# UT_0.4.2 Trabajo con formularios

Eventos de formulario y de la interfaz, delegación de eventos y trabajo con formularios

*Los formularios y sus campos (cuadros de texto, listas, casillas...) son una parte esencial de las aplicaciones web interactivas. Los formularios funcionan sin JavaScript, pero con él ganan mucha interactividad y facilidad de uso. En este tema aprenderás a reaccionar a lo que hace el usuario con ellos, a leer y enviar sus datos y a validarlos.*

Este tema se organiza en cinco bloques:

- **Sección 1.** Los eventos propios de los formularios, del foco y de la interfaz de usuario (continuación del tema anterior).
- **Sección 2.** La *delegación de eventos*: una técnica para escribir menos código y que funcione con elementos creados más tarde.
- **Sección 3.** Cómo acceder a formularios y campos, y leer sus valores.
- **Sección 4.** Cómo enviar y restablecer un formulario mediante programación.
- **Sección 5.** Cómo validar los datos: validación nativa de HTML5 y API de validación de restricciones.

> **Cómo ejecutar los ejemplos**
>
> En todos los ejemplos el código JavaScript se carga como **módulo ES6** (`<script type="module" src="...">`). Los módulos se ejecutan cuando el HTML ya está analizado (como si llevaran `defer`), por lo que **no necesitas** envolver el código en `DOMContentLoaded`. Además, los módulos trabajan en modo estricto y, al cargarlos desde un archivo `.js` externo, hay que servir la página desde un servidor (por ejemplo, la extensión *Live Server* de VS Code); abriéndola directamente con `file://` el navegador bloquea el módulo.

# Índice de contenidos

1 Eventos de formulario y de la interfaz de usuario ... 3
1.1 Eventos al trabajar con formularios ... 3
1.2 Eventos al enfocar elementos ... 4
1.3 Eventos generales de la interfaz de usuario ... 5
2 Delegación de eventos ... 7
2.1 El problema: un escuchador por elemento ... 7
2.2 La idea: aprovechar el burbujeo ... 8
2.3 Paso a paso ... 9
2.4 Varias acciones con un solo escuchador ... 11
2.5 Eventos que no burbujean ... 11
2.6 Errores frecuentes ... 12
3 Acceso a formularios y campos de formulario ... 12
3.1 Acceso a formularios ... 12
3.2 Acceso a los elementos del formulario ... 14
3.3 Lectura del valor de campos de texto y de contraseña ... 15
3.4 Lectura del valor de las casillas de verificación ... 16
3.5 Lectura del valor de los botones de opción ... 16
3.6 Lectura del valor de las listas de selección ... 17
3.7 Lectura de los valores de listas de selección múltiples ... 18
3.8 Rellenar listas de selección con valores mediante JavaScript ... 19
4 Envío y restablecimiento de formularios mediante programación ... 19
5 Validación de formularios ... 22
5.1 Validación nativa de HTML5 ... 22
5.2 La API de validación de restricciones ... 25
5.3 Mensajes de validación personalizados ... 27
5.4 Decidir cuándo y cómo se muestran los errores ... 29
5.5 Ejemplo completo con módulos ... 29
6 Resumen ... 30

# 1 Eventos de formulario y de la interfaz de usuario

Continuamos con el catálogo de eventos del tema anterior. Aquí veremos los eventos específicos de los formularios, los del foco y otros eventos generales de la interfaz de usuario.

## 1.1 Eventos al trabajar con formularios

Los formularios disponen de eventos propios: se activan cuando el usuario modifica un `<input>` o `<textarea>`, cambia una lista desplegable, una casilla o un botón de opción, cuando un campo no supera la validación nativa o cuando se envía o se restablece el formulario. Casi todos son de tipo `Event`; las excepciones son `input` y `beforeinput` (`InputEvent`) y `submit` (`SubmitEvent`) (véase Tabla 1).

| Evento | Descripción | Tipo de objeto |
|---|---|---|
| `input` | Cambia el valor de un `<input>` o `<textarea>` (o de un elemento con `contenteditable`). Se activa con cada modificación. | InputEvent |
| `beforeinput` | Justo antes de modificar el valor del campo. Es cancelable. | InputEvent |
| `change` | Se confirma un cambio: al perder el foco un campo de texto, o al instante en casillas, botones de opción y listas. | Event |
| `submit` | Se envía el formulario (botón de envío, tecla Intro o `requestSubmit()`). Se activa en el `<form>`. | SubmitEvent |
| `reset` | Se restablece el formulario (botón de restablecimiento o `reset()`). | Event |
| `invalid` | Un campo no supera la validación nativa del navegador. | Event |

***Tabla 1*** *Eventos al interactuar con campos y formularios*

### input, change y submit en la práctica

- `input` se activa con cada modificación (cada tecla, al pegar, al borrar...). `change` se activa cuando el valor se **confirma**: al perder el foco en un campo de texto e inmediatamente en casillas y listas.
- `submit` se activa en el elemento `<form>` (no en el botón), tanto al pulsar un botón de envío como al pulsar Intro. En `evento.submitter` está el botón que lo provocó.
- `beforeinput` se activa justo antes de modificar el valor y es cancelable: permite impedir ciertos caracteres.

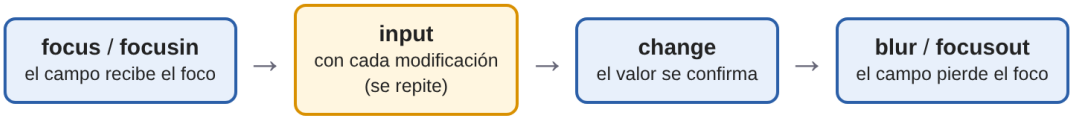



***Figura 1*** *Orden habitual de los eventos de un campo de texto y del formulario*

El Listado 1 muestra un formulario mínimo y el Listado 2 escucha sus tres eventos principales. Prueba a escribir, salir del campo y pulsar el botón mientras observas la consola.

```
<form id="contacto">
  <label for="nombre">Nombre:</label>
  <input type="text" id="nombre" name="nombre">
  <button type="submit">Enviar</button>
</form>
<script type="module" src="js/principal.js"></script>
```

***Listado 1*** *HTML de un formulario de contacto mínimo*

```
const formulario = document.getElementById('contacto');
const campoNombre = document.getElementById('nombre');

// input: con cada modificación (cada tecla, al pegar, al borrar...)
campoNombre.addEventListener('input', (evento) => {
  console.log(`Escribiendo: ${evento.target.value}`);
});

// change: cuando el valor se confirma (al perder el foco o pulsar Intro)
campoNombre.addEventListener('change', (evento) => {
  console.log(`Valor confirmado: ${evento.target.value}`);
});

// submit: se escucha en el <form>, no en el botón
formulario.addEventListener('submit', (evento) => {
  evento.preventDefault(); // evita el envío y la recarga de la página
  const datos = Object.fromEntries(new FormData(formulario));
  console.log(datos);            // { nombre: 'Ana' }
  console.log(evento.submitter); // el <button> que provocó el envío
});
```

***Listado 2*** *Eventos input, change y submit de un formulario*

En el Listado 3 se usa `beforeinput` para impedir que se escriban cifras. Como `evento.data` contiene el texto que se va a insertar (y es `null` al borrar), basta comprobarlo con una expresión regular y cancelar el evento.

```
// Impide escribir cifras en el campo «nombre»
campoNombre.addEventListener('beforeinput', (evento) => {
  if (evento.data && /\d/.test(evento.data)) {
    evento.preventDefault(); // cancela la modificación
  }
});
```

***Listado 3*** *Cancelar la escritura de cifras con beforeinput*

> **Novedades útiles**
>
> Para enviar por código usa `formulario.requestSubmit()`: valida el formulario y lanza el evento `submit`. El método clásico `formulario.submit()` envía **sin validar y sin lanzar `submit`** (se explica en la sección 4). Y `new FormData(formulario)` recoge todos los campos con atributo `name` en una sola línea (sección 4).

> **Nota**
>
> El funcionamiento detallado de los formularios se explica en las secciones 3, 4 y 5 de este tema.

## 1.2 Eventos al enfocar elementos

Un elemento «tiene el foco» cuando está listo para recibir la entrada del usuario (por ejemplo, tras hacer clic en él o navegar hasta él con la tecla Tab). Cada vez que un elemento gana o pierde el foco se activa un evento de tipo `FocusEvent` (Tabla 2).

| Evento | Descripción | Tipo de objeto |
|---|---|---|
| `focus` | El elemento recibe el foco. **No** burbujea. | FocusEvent |
| `blur` | El elemento pierde el foco. **No** burbujea. | FocusEvent |
| `focusin` | El elemento recibe el foco. A diferencia de `focus`, **sí** burbujea (Tabla 4). | FocusEvent |
| `focusout` | El elemento pierde el foco. A diferencia de `blur`, **sí** burbujea. | FocusEvent |

***Tabla 2*** *Eventos al enfocar elementos*

Como `focusin` y `focusout` burbujean, un único escuchador en el formulario atiende a todos sus campos (Listado 5). Aquí se resalta el campo activo añadiéndole la clase `activo`; verás el resultado en la Figura 2.

```
<form id="registro">
  <input type="text" name="usuario" placeholder="Usuario">
  <input type="email" name="correo" placeholder="Correo">
</form>
```

***Listado 4*** *HTML de un formulario de registro con dos campos*

```
const formulario = document.getElementById('registro');

// focusin y focusout burbujean: basta un único escuchador en el formulario
formulario.addEventListener('focusin', (evento) => {
  evento.target.classList.add('activo');
});
formulario.addEventListener('focusout', (evento) => {
  evento.target.classList.remove('activo');
});
```

***Listado 5*** *Resaltar el campo que tiene el foco con focusin y focusout*




***Figura 2*** *El campo que tiene el foco aparece resaltado*

> **Recuerda**
>
> Los eventos `blur` y `change` sirven para validar un campo cuando el usuario termina de escribir en él. Lo verás en la sección 5.

> **Novedad CSS**
>
> Si solo necesitas cambiar el **aspecto** del campo enfocado no hace falta JavaScript: las pseudoclases CSS `:focus` y `:focus-within` (esta última se aplica a un contenedor si alguno de sus descendientes tiene el foco) lo resuelven.

```
/* Si solo quieres cambiar estilos, CSS lo resuelve sin JavaScript */
input:focus { outline: 2px solid royalblue; }
form:focus-within { background: #eef4ff; }
```

***Listado 6*** *Resaltado del foco solo con CSS*

## 1.3 Eventos generales de la interfaz de usuario

Además de los eventos vistos hasta ahora hay otros que pueden clasificarse como eventos de la interfaz de usuario (la mayoría, aunque no todos, son de tipo `UIEvent`). La Tabla 3 ofrece una visión general.

| Evento | Descripción | Tipo de objeto |
|---|---|---|
| `load` | La página y todos sus recursos (imágenes, hojas de estilo...) se han cargado por completo. | Event |
| `DOMContentLoaded` | El árbol DOM está completo (no espera a imágenes ni estilos). Con módulos no suele ser necesario. | Event |
| `unload` | **Obsoleto**: poco fiable y en retirada. Utiliza `pagehide` o `visibilitychange`. | Event |

| Evento | Descripción | Tipo de objeto |
|---|---|---|
| `beforeunload` | Justo antes de descargar la página. Sirve para avisar de cambios sin guardar; los navegadores no muestran mensajes personalizados y exigen que el usuario haya interactuado con la página. | BeforeUnloadEvent |
| `pagehide` | El usuario abandona la página (sustituye a `unload`). | PageTransitionEvent |
| `visibilitychange` | La pestaña pasa a estar visible u oculta (véase `document.visibilityState`). | Event |
| `abort` | Se cancela la carga de un recurso. | Event |
| `error` | Falla la carga de un recurso o se produce un error de JavaScript (en `window`). | Event / ErrorEvent |
| `resize` | Cambia el tamaño de la ventana del navegador. | UIEvent |
| `scroll` | El usuario se desplaza (en la página o en un elemento). | UIEvent |
| `scrollend` | Termina un desplazamiento. Disponible en los navegadores actuales (compruébalo en caniuse.com). | Event |
| `select` | Se selecciona texto en un `<input>` o `<textarea>`. | Event |
| `cut` | Se corta contenido de un campo (por ejemplo, con Ctrl + X). | ClipboardEvent |
| `copy` | Se copia contenido de un campo (por ejemplo, con Ctrl + C). | ClipboardEvent |
| `paste` | Se pega contenido en un campo (por ejemplo, con Ctrl + V). | ClipboardEvent |

***Tabla 3*** *Descripción general de los eventos de la interfaz de usuario*

### Uso de los eventos load y scroll

El evento `load` se utiliza para ejecutar código inicial cuando la página ya está cargada. Un caso de uso típico del evento `scroll` es mostrar elementos de la página según la posición de desplazamiento vertical, un efecto muy habitual en las páginas de una sola página: al bajar, aparecen poco a poco textos o imágenes.

El ejemplo siguiente muestra un texto cuando el usuario se desplaza. El Listado 7 contiene la estructura: el `<span>` con el texto *Juan Pérez* está oculto al cargar. El Listado 8 escucha `scroll` en `window` y, con `window.scrollY` (posición vertical), añade o quita la clase `visible` cuando se supera un umbral (10 píxeles). El Listado 9 hace que el elemento aparezca de forma gradual.

```
<body>
  <div id="contenido">
    Hola
    <span id="nombre" class="oculto">Juan Pérez</span>
  </div>
  <div class="relleno"></div>
  <script type="module" src="js/principal.js"></script>
</body>
```

***Listado 7*** *HTML para mostrar un elemento según el desplazamiento*

```
const elementoNombre = document.getElementById('nombre');

window.addEventListener('scroll', () => {
  // toggle(clase, condición): añade la clase si la condición es true y la quita si es false
  elementoNombre.classList.toggle('visible', window.scrollY > 10);
}, { passive: true });
```

***Listado 8*** *Mostrar un elemento según la posición de desplazamiento*

```
.oculto { opacity: 0; transition: opacity 1s; }
.oculto.visible { opacity: 1; }
.relleno { height: 200vh; } /* Alarga la página para poder desplazarse */
```

***Listado 9*** *CSS de las clases oculto, visible y relleno*

#### Alternativa moderna: IntersectionObserver

El evento `scroll` se dispara muchísimas veces por segundo (por eso se registra con `{ passive: true }`, que avisa al navegador de que no se llamará a `preventDefault()`). Para saber si un elemento ha entrado en la zona visible es más sencillo y eficiente usar `IntersectionObserver`, que avisa solo cuando cambia la visibilidad del elemento (Listado 10). Si lo que quieres es reaccionar cuando el desplazamiento ha terminado, existe además el evento `scrollend`.

```
const observador = new IntersectionObserver((entradas) => {
  entradas.forEach((entrada) => {
    if (entrada.isIntersecting) {
      entrada.target.classList.add('visible');
      observador.unobserve(entrada.target); // solo la primera vez
    }
  });
}, { threshold: 0.2 }); // se avisa cuando se ve el 20 % del elemento

document.querySelectorAll('.oculto').forEach((elemento) => {
  observador.observe(elemento);
});
```

***Listado 10*** *Mostrar elementos al entrar en pantalla con IntersectionObserver*

#### Aviso de cambios sin guardar con beforeunload

Un uso muy práctico en formularios: avisar al usuario si intenta cerrar la página con datos sin enviar (Listado 11). Se llama a `preventDefault()` y el navegador muestra su propio mensaje genérico.

```
const formulario = document.getElementById('contacto');
let hayCambios = false;

formulario.addEventListener('input', () => { hayCambios = true; });
formulario.addEventListener('submit', () => { hayCambios = false; });

window.addEventListener('beforeunload', (evento) => {
  if (hayCambios) evento.preventDefault(); // el navegador muestra su propio aviso
});
```

***Listado 11*** *Avisar de cambios sin guardar*

# 2 Delegación de eventos

Hasta ahora hemos registrado un escuchador en cada elemento que nos interesaba. Funciona, pero con listas largas o con elementos que se crean más tarde se vuelve incómodo e ineficiente. La *delegación de eventos* resuelve ambos problemas apoyándose en lo que ya sabes del burbujeo: **cuando haces clic en un elemento, el evento sube por sus ancestros**.

## 2.1 El problema: un escuchador por elemento

Imagina una lista de tareas con un botón *Borrar* en cada una y un botón *Añadir tarea* (Listado 12 y Figura 3).

```
<ul id="tareas">
  <li>Estudiar eventos <button class="borrar">Borrar</button></li>
  <li>Hacer el ejercicio <button class="borrar">Borrar</button></li>
</ul>
<button id="agregar">Añadir tarea</button>
```

***Listado 12*** *HTML de la lista de tareas*

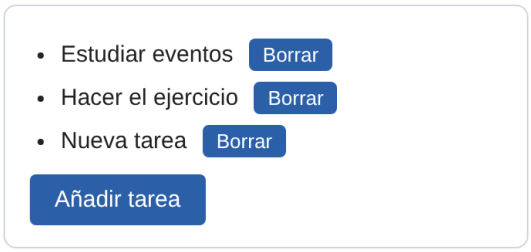



***Figura 3*** *Lista de tareas con botones para borrar*

La solución «obvia» es registrar un escuchador en cada botón (Listado 13). Pero tiene un fallo: el escuchador solo se añade a los botones que **existen en ese momento**. Al pulsar *Añadir tarea* se crea un `<li>` con su botón, y ese botón no responde al clic porque nadie le ha registrado el escuchador.

```
const lista = document.getElementById('tareas');

// Un escuchador por cada botón que existe AHORA
document.querySelectorAll('.borrar').forEach((boton) => {
  boton.addEventListener('click', () => {
    boton.closest('li').remove();
  });
});

document.getElementById('agregar').addEventListener('click', () => {
  const tarea = document.createElement('li');
  tarea.innerHTML = 'Nueva tarea <button class="borrar">Borrar</button>';
  lista.append(tarea); // ¡El botón de esta tarea NO tiene escuchador!
});
```

***Listado 13*** *Un escuchador por botón: las tareas nuevas no funcionan*

> **Importante**
>
> Con este enfoque habría que volver a registrar el escuchador cada vez que se crea una tarea, y además se acumulan tantos escuchadores como botones haya (con mil elementos, mil escuchadores).

## 2.2 La idea: aprovechar el burbujeo

Si el evento sube desde el botón hasta sus ancestros, no hace falta escuchar en el botón: podemos escuchar **una sola vez en un ancestro** (aquí, el `<ul>`) y averiguar, mediante `evento.target`, qué elemento originó el evento. A esto se le llama *delegar* el evento en el ancestro. La Figura 4 compara ambos enfoques.

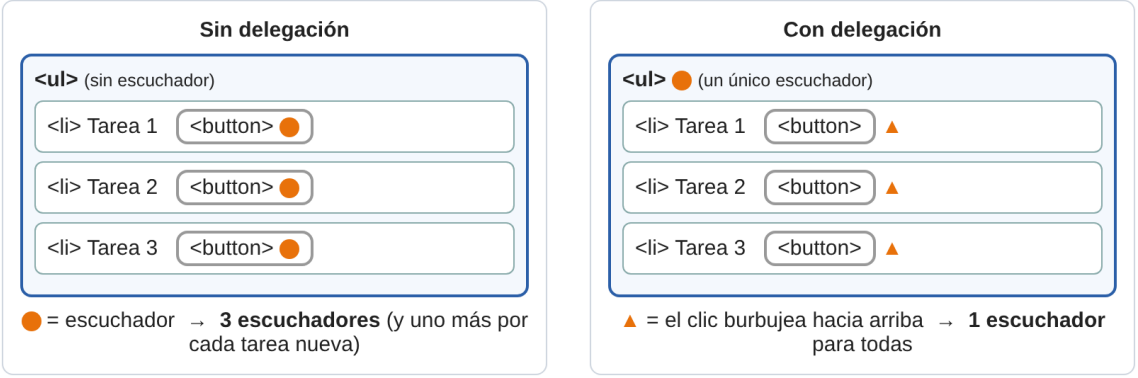



***Figura 4*** *Sin delegación hay un escuchador por elemento; con delegación, uno solo en el ancestro*

Ventajas: (1) **menos escuchadores** (uno en lugar de cientos) y (2) **funciona con elementos añadidos después**, porque el escuchador está en el ancestro, que ya existía.

## 2.3 Paso a paso

Vamos a construir la solución en seis pasos. La Figura 5 resume lo que ocurre cuando se hace clic en un botón.

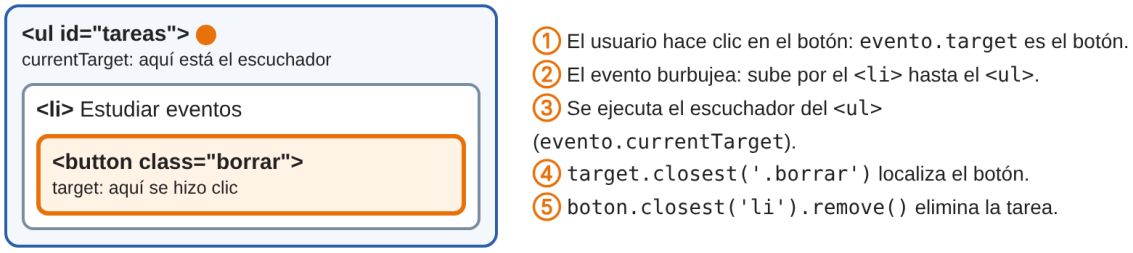



***Figura 5*** *Recorrido de un clic en una lista con delegación de eventos*

### Pasos 1 a 3: ancestro, escuchador y origen del evento

- **Paso 1. Elige el ancestro.** Debe contener a todos los elementos que te interesan y existir desde el principio (y no ser eliminado). Aquí, la lista `<ul id="tareas">`. Cuanto más cercano a los elementos, mejor: evita delegar en `document` o `body` salvo que sea necesario.
- **Paso 2. Registra un único escuchador** en ese ancestro, con el evento que te interesa (`click`).
- **Paso 3. Averigua quién originó el evento.** El objeto de evento tiene dos propiedades que conviene no confundir: `evento.target` (el elemento en el que realmente se produjo el evento, el más profundo) y `evento.currentTarget` (el elemento que tiene el escuchador, aquí siempre el `<ul>`).

```
// Paso 1: el ancestro común (existe desde el principio y no se elimina)
const lista = document.getElementById('tareas');

// Paso 2: un único escuchador en él
lista.addEventListener('click', (evento) => {
  // Paso 3: ¿quién originó el evento?
  console.log(evento.target);        // el elemento pulsado: <button>, <li> o <ul>
  console.log(evento.currentTarget); // el elemento que tiene el escuchador: siempre <ul>
});
```

***Listado 14*** *Pasos 1 a 3: escuchador único y origen del evento*

Prueba este código y haz clic en un botón, en el texto de un `<li>` y en el hueco del `<ul>`: `currentTarget` siempre será el `<ul>`, pero `target` cambia según dónde hayas hecho clic. Por eso el siguiente paso es **filtrar**: nuestro escuchador se ejecuta con *cualquier* clic dentro de la lista y solo queremos actuar si el clic fue en un botón *Borrar*.

### Pasos 4 a 6: filtrar, comprobar y actuar

- **Paso 4. Filtra con `closest()`.** `elemento.closest(selector)` sube desde el elemento hasta encontrar el primer ancestro (o él mismo) que cumple el selector, y devuelve `null` si no hay ninguno. Si es `null`, el clic no fue en un botón *Borrar* y el escuchador termina con `return`.
- **Paso 5. Comprueba que está dentro de tu ancestro** con `lista.contains(boton)`. Es una salvaguarda para cuando hay contenedores anidados y `closest()` pudiera encontrar un elemento que está fuera de la lista.
- **Paso 6. Actúa.** Con el botón localizado, `boton.closest('li')` te da la tarea a la que pertenece y `remove()` la elimina.

```
lista.addEventListener('click', (evento) => {
  // Paso 4: ¿el elemento pulsado (o uno de sus ancestros) es un botón «borrar»?
  const boton = evento.target.closest('.borrar');
  if (!boton) return; // el clic fue en otra parte: no hacemos nada

  // Paso 5: ¿ese botón está dentro de nuestra lista?
  if (!lista.contains(boton)) return;

  // Paso 6: actuar
  boton.closest('li').remove();
});
```

***Listado 15*** *Pasos 4 a 6: filtrar, comprobar y actuar*

> **Un detalle importante**
>
> **¿Por qué `closest()` y no `target.matches()`?** Si el botón contiene otros elementos (un icono, un `<span>`...), `evento.target` será ese elemento interior y no el botón. `matches('.borrar')` devolvería `false`, mientras que `closest('.borrar')` encuentra el botón se pulse donde se pulse dentro de él.

```
<button class="borrar"><span class="icono">🗑</span> Borrar</button>
```

***Listado 16*** *Botón con un elemento interior*

```
evento.target.matches('.borrar');   // false si se pulsa el <span>: target es el <span>
evento.target.closest('.borrar');   // el <button>, se pulse donde se pulse dentro de él
```

***Listado 17*** *target.matches() frente a target.closest()*

Juntando todo, el Listado 18 sustituye al del apartado 2.1. Ahora las tareas nuevas funcionan sin registrar nada más (**Paso 7: pruébalo**: añade tareas y bórralas).

```
const lista = document.getElementById('tareas');

// Un único escuchador atiende a todos los botones, presentes y futuros
lista.addEventListener('click', (evento) => {
  const boton = evento.target.closest('.borrar');
  if (!boton || !lista.contains(boton)) return;
  boton.closest('li').remove();
});

// Las tareas nuevas funcionan sin registrar nada más
document.getElementById('agregar').addEventListener('click', () => {
  const tarea = document.createElement('li');
  tarea.innerHTML = 'Nueva tarea <button class="borrar">Borrar</button>';
  lista.append(tarea);
});
```

***Listado 18*** *Delegación de eventos con un único escuchador en el elemento padre*

## 2.4 Varias acciones con un solo escuchador

Un mismo escuchador puede atender botones con distintas funciones. Se identifica la acción con un atributo `data-*` (por ejemplo, `data-accion="borrar"`), que se lee en JavaScript con `elemento.dataset.accion`. En el Listado 20 cada acción es una función guardada en un objeto, y el valor de `data-accion` es el nombre de la función que hay que ejecutar.

```
<ul id="tareas">
  <li>Estudiar eventos
    <button data-accion="completar">Hecha</button>
    <button data-accion="borrar">Borrar</button>
  </li>
</ul>
```

***Listado 19*** *Botones con el atributo data-accion*

```
const lista = document.getElementById('tareas');

// Cada acción es una función; el nombre coincide con el valor de data-accion
const acciones = {
  completar: (tarea) => tarea.classList.toggle('hecha'),
  borrar: (tarea) => tarea.remove(),
};

lista.addEventListener('click', (evento) => {
  const boton = evento.target.closest('button[data-accion]');
  if (!boton || !lista.contains(boton)) return;

  const accion = acciones[boton.dataset.accion]; // dataset.accion lee data-accion
  if (accion) accion(boton.closest('li'));
});
```

***Listado 20*** *Un escuchador, varias acciones*

## 2.5 Eventos que no burbujean

La delegación solo funciona con eventos que burbujean. Algunos no lo hacen (Tabla 4); en esos casos se usa un evento equivalente que sí burbujea o se escucha en la fase de captura (tercer argumento `true` de `addEventListener`).

| Evento sin burbujeo | Alternativa para delegar |
|---|---|
| focus / blur | `focusin` / `focusout` |
| mouseenter / mouseleave | `mouseover` / `mouseout` |
| invalid | Escuchar en fase de captura (`true`) |
| load / error / scroll (en elementos) | Escuchar en fase de captura (`true`) |

***Tabla 4*** *Eventos que no burbujean y alternativas*

Un ejemplo muy útil es validar cada campo de un formulario cuando el usuario lo abandona, con un único escuchador (Listado 21). Si prefieres usar `blur`, hay que capturarlo (Listado 22).

```
const formulario = document.getElementById('registro');

// Un único escuchador valida cualquier campo cuando el usuario lo abandona
formulario.addEventListener('focusout', (evento) => {
  const campo = evento.target;
  if (campo.matches('input')) {
    console.log(`${campo.name}: ${campo.validity.valid ? 'válido' : 'no válido'}`);
  }
});
```

***Listado 21*** *Delegar focusout en un formulario*

```
// Alternativa: escuchar en la fase de captura (tercer argumento true)
formulario.addEventListener('blur', (evento) => {
  console.log(`${evento.target.name} ha perdido el foco`);
}, true);
```

***Listado 22*** *Escuchar blur en la fase de captura*

### 2.6 Errores frecuentes

| Error | Consecuencia y solución |
|---|---|
| Usar solo `evento.target` | Si el elemento tiene hijos, `target` puede ser un hijo. Usa `closest()`. |
| No filtrar el elemento | El escuchador actúa con cualquier clic dentro del ancestro. Comprueba siempre `if (!boton) return;`. |
| Llamar a `stopPropagation()` en un hijo | El evento no llega al ancestro y la delegación deja de funcionar. |
| Delegar siempre en `document` | Todo el documento pasa por tu escuchador. Elige el ancestro más cercano. |
| Delegar eventos que no burbujean | No se ejecuta nunca. Usa `focusin`/`focusout` o la fase de captura. |
| Delegar eventos muy frecuentes (`mousemove`) | Se ejecuta demasiadas veces. Pon un escuchador directo en el elemento concreto. |

***Tabla 5*** *Errores frecuentes al delegar eventos*

## 3 Acceso a formularios y campos de formulario

Ya sabes utilizar JavaScript para acceder a elementos de una página. El acceso a formularios funciona de manera muy similar, con algunas particularidades.

### 3.1 Acceso a formularios

Tienes varias opciones para acceder a un formulario:

- Los métodos de selección del DOM: `getElementById()`, `getElementsByClassName()`, `getElementsByTagName()`, `getElementsByName()`, `querySelector()` y `querySelectorAll()`.
- La propiedad `forms` del objeto `document`.
- Una propiedad de `document` con el nombre del formulario.

Lo demostraremos con un sencillo formulario de inicio de sesión: un campo de usuario, uno de contraseña, una casilla para recordar los datos y un botón de envío (Listado 23 y Figura 6).

```
<form id="acceso" name="acceso" method="post" action="acceso.html">
  <div>
    <label for="usuario">Usuario:</label>
    <input type="text" id="usuario" name="usuario" placeholder="Usuario">
  </div>
  <div>
    <label for="clave">Contraseña:</label>
    <input type="password" id="clave" name="clave" placeholder="Contraseña">
  </div>
  <div>
    <label>
      <input type="checkbox" id="recordar" name="recordar" value="on">
      Recordarme en este equipo
    </label>
  </div>
  <button type="submit" id="entrar">Entrar</button>
</form>
<script type="module" src="js/principal.js"></script>
```

***Listado 23*** *HTML de un formulario de inicio de sesión sencillo*

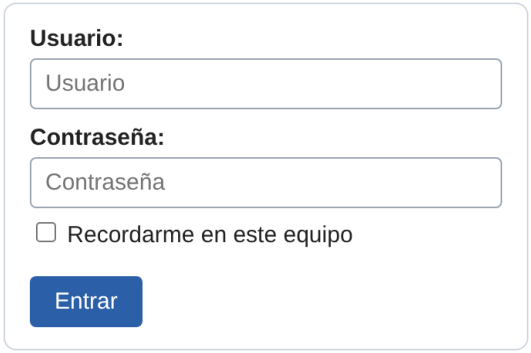



***Figura 6*** *Formulario de inicio de sesión sencillo*

El Listado 24 muestra las tres formas de acceso. La propiedad `forms` contiene la lista de formularios de la página en el orden en que aparecen (el primero en la posición 0, el segundo en la 1...). Además, todo formulario con atributo `name` crea una propiedad del mismo nombre en `document`: aquí, `document.acceso`.

```
// 1) Métodos de selección del DOM (la opción recomendada)
const formPorId = document.getElementById('acceso');
const formPorNombre = document.getElementsByName('acceso')[0];
const formPorSelector = document.querySelector('form');

// 2) Propiedad forms de document: por posición
const formPorPosicion = document.forms[0];

// 3) Por el nombre del formulario
const formPorNombreDoc = document.acceso;   // equivale a document.forms.acceso

console.log(formPorId.id);         // «acceso»
console.log(formPorNombre.id);     // «acceso»
console.log(formPorSelector.id);   // «acceso»
console.log(formPorPosicion.id);   // «acceso»
console.log(formPorNombreDoc.id);  // «acceso»
```

***Listado 24*** *Acceso a un formulario*

**¿Cuál elegir?** Utiliza siempre los métodos de selección del DOM, preferiblemente `getElementById()` o `querySelector()`:

- `document.forms[n]` obliga a conocer la posición exacta del formulario; si se añade o elimina otro formulario anterior, hay que corregir el índice.
- `document.nombre` puede chocar con propiedades y métodos que `document` ya tiene (por ejemplo, un formulario llamado `title` o `body`).
- `getElementById()` y `querySelector()` identifican el elemento sin ambigüedad y no obligan a tocar el código si cambia el HTML.

Los formularios tienen propiedades como `elements` (sus controles, ver apartado 3.2), `length` (número de controles), `name`, `action` y `method` (a dónde y cómo se envían los datos: `post` o `get`) (Listado 25).

```
const formulario = document.getElementById('acceso');
console.log(formulario.elements); // controles del formulario (apartado 3.2)
console.log(formulario.length);   // 4 controles: 3 campos y 1 botón
console.log(formulario.name);     // «acceso»
console.log(formulario.action);   // URL completa del atributo action
console.log(formulario.method);   // «post»
```

***Listado 25*** *Propiedades de un formulario*

## 3.2 Acceso a los elementos del formulario

Para acceder a un control del formulario puedes usar los métodos de selección del DOM o la propiedad `elements` del formulario, una colección con todos sus controles. Dentro de `elements` se accede al control por su **posición** (el orden en que aparece en el formulario) o por su **nombre** (atributo `name` o `id`).

```
// 1) Métodos de selección del DOM
const campoUsuario = document.getElementById('usuario');
const campoClave = document.getElementById('clave');
const casillaRecordar = document.getElementById('recordar');
const botonEntrar = document.getElementById('entrar');

// 2) Propiedad elements, por posición...
const formulario = document.getElementById('acceso');
const controles = formulario.elements;
console.log(controles.length);   // 4
console.log(controles[0].id);    // «usuario»
console.log(controles[3].id);    // «entrar»

// 3) ...o por nombre (más legible y no depende del orden)
console.log(controles.usuario === campoUsuario); // true
console.log(controles.clave.id);                 // «clave»
```

***Listado 26*** *Acceso a los elementos de un formulario*

Igual que antes, el acceso con los métodos del DOM es el más resistente a cambios del HTML. El acceso por posición es el menos recomendable (si cambia el orden de los campos, hay que rehacer el código). El acceso por nombre, `formulario.elements.usuario`, es cómodo y claro.

Cada elemento del formulario se representa con un tipo de objeto especial (Tabla 6): campos de texto, de contraseña, botones de opción y casillas son `HTMLInputElement`; las áreas de texto, `HTMLTextAreaElement`; las listas, `HTMLSelectElement`; y sus opciones, `HTMLOptionElement`.

| Elemento del formulario | Tipo de objeto |
|---|---|
| Formulario (`<form>`) | HTMLFormElement |
| Campo de texto (`<input type="text">`) | HTMLInputElement |
| Campo de contraseña (`<input type="password">`) | HTMLInputElement |
| Botón de opción (`<input type="radio">`) | HTMLInputElement |
| Casilla de verificación (`<input type="checkbox">`) | HTMLInputElement |
| Área de texto (`<textarea>`) | HTMLTextAreaElement |
| Lista de selección (`<select>`) | HTMLSelectElement |
| Opción de una lista (`<option>`) | HTMLOptionElement |
| Botón (`<button>`) | HTMLButtonElement |

***Tabla 6*** *Elementos de formulario y su tipo de objeto*

> **Objetos JavaScript para componentes HTML**
>
> Cada componente HTML está representado por un objeto JavaScript distinto, con propiedades y métodos específicos. En los apartados siguientes veremos los de cada tipo de control.

## 3.3 Lectura del valor de campos de texto y de contraseña

El valor de un campo se obtiene con la propiedad `value`. En el Listado 27 se registra un escuchador para el evento `change` en los dos campos. Dentro de un escuchador definido con `function`, la palabra clave `this` es el elemento que tiene el escuchador. Fíjate también en que el valor de un campo de contraseña se guarda como **texto plano**, sin cifrar.

```
const campoUsuario = document.getElementById('usuario');
const campoClave = document.getElementById('clave');

campoUsuario.addEventListener('change', function () {
  console.log(this.value); // this es el elemento que tiene el escuchador
});
campoClave.addEventListener('change', function () {
  console.log(this.value);
});
```

***Listado 27*** *Acceso a los valores de los campos de texto y contraseña con this*

**Cuidado con las funciones flecha:** en ellas `this` no es el elemento, sino el `this` del contexto en el que se escribió la función. En un módulo ese `this` vale `undefined`, así que `this.value` produce un error (Listado 28). Hay dos alternativas: usar la variable que guarda el elemento, o usar el objeto de evento (`evento.target`) (Listado 29).

```
campoUsuario.addEventListener('change', () => {
  console.log(campoUsuario.value); // ✓ variable externa: valor introducido
  console.log(this.value);         // ✗ error: en un módulo, this vale undefined
});
```

***Listado 28*** *En una función flecha this no es el elemento*

```
campoUsuario.addEventListener('change', (evento) => {
  console.log(evento.target.value); // ✓ valor introducido
});
campoClave.addEventListener('change', (evento) => {
  console.log(evento.target.value);
});
```

***Listado 29*** *Acceso a través del objeto de evento*

> **Atención**
>
> `value` es **siempre texto**, incluso en un `<input type="number">`. Para operar con él, usa `valueAsNumber` (o conviértelo con `Number()`).

```
const campoEdad = document.getElementById('edad'); // <input type="number"> con el valor 18

console.log(campoEdad.value + 1);         // «181»: value siempre es texto
console.log(campoEdad.valueAsNumber + 1); // 19: valueAsNumber ya es un número
```

***Listado 30*** *value es texto, valueAsNumber es un número*

## 3.4 Lectura del valor de las casillas de verificación

Las casillas se leen de forma parecida. Su `value` vale `on` por defecto, pero lo que interesa es saber si están marcadas: para eso se usa la propiedad booleana `checked` (Listado 31).

```
const casilla = document.getElementById('recordar');

casilla.addEventListener('change', () => {
  console.log(casilla.value);   // «on» por defecto
  console.log(casilla.checked); // true o false
});
```

***Listado 31*** *Acceso a una casilla de verificación*

## 3.5 Lectura del valor de los botones de opción

Los botones de opción aparecen siempre en grupo, y dentro de un grupo solo se puede marcar uno. El grupo lo determina el atributo `name`: los botones del Listado 32 pertenecen todos al grupo `pizza` (Figura 7).

```
<form id="pedido" name="pedido">
  <label><input type="radio" name="pizza" value="P001"> Pizza Salami</label>
  <label><input type="radio" name="pizza" value="P002"> Pizza Margarita</label>
  <label><input type="radio" name="pizza" value="P003"> Pizza Atún</label>
  <label><input type="radio" name="pizza" value="P004"> Pizza Mozzarella</label>
  <label><input type="radio" name="pizza" value="P005"> Pizza Hawaiana</label>
</form>
<p>Selección: <span id="seleccion"></span></p>
```

***Listado 32*** *HTML para un grupo de botones de opción*




***Figura 7*** *Un grupo de botones de opción*

La forma más sencilla de acceder a los botones es `formulario.elements.pizza`, que devuelve la lista de todos los botones del grupo. En el Listado 33 se recorre esa lista y se registra un escuchador de `change` en cada botón; al marcar uno, su valor se escribe en el `<span>` de la selección.

```
const formulario = document.getElementById('pedido');
const mensaje = document.getElementById('seleccion');
const radios = formulario.elements.pizza; // los 5 botones del grupo «pizza»

for (const radio of radios) {
  radio.addEventListener('change', (evento) => {
    console.log(evento.target.value);              // p. ej., «P002»
    mensaje.textContent = evento.target.value;
  });
}
```

***Listado 33*** *Acceso a los botones de opción de un grupo*

**Una forma más corta**

La lista que devuelve `formulario.elements.pizza` (un `RadioNodeList`) tiene una propiedad `value` con el valor del botón **marcado** (o `''` si no hay ninguno). Además, como `change` burbujea, basta un único escuchador en el formulario: es delegación de eventos aplicada a formularios.

```
// El evento change burbujea hasta el formulario: basta un único escuchador
formulario.addEventListener('change', () => {
  // La propiedad value del grupo devuelve el valor del botón marcado ('' si no hay ninguno)
  mensaje.textContent = formulario.elements.pizza.value;
});
```

***Listado 34*** *Un único escuchador y la propiedad value del grupo*

## 3.6 Lectura del valor de las listas de selección

Las listas de selección también reaccionan al evento `change`, que se activa cada vez que se elige una opción. Partimos del Listado 35.

```
<form id="pedido" name="pedido">
  <select id="pizza" name="pizza">
    <option value="P001" selected>Pizza Salami</option>
    <option value="P002">Pizza Margarita</option>
    <option value="P003">Pizza Atún</option>
    <option value="P004">Pizza Mozzarella</option>
    <option value="P005">Pizza Hawaiana</option>
  </select>
</form>
<p>Selección: <span id="seleccion"></span></p>
```

***Listado 35*** *HTML de la lista de selección*

En el Listado 36, `value` devuelve el `value` de la opción elegida. Además, `selectedIndex` indica la posición de la opción elegida y `options[indice]` da acceso al elemento `<option>` (`HTMLOptionElement`), que tiene sus propias propiedades `value` y `text` (el texto que ve el usuario). El resultado aparece en la Figura 8.

```
const lista = document.getElementById('pizza');
const mensaje = document.getElementById('seleccion');

lista.addEventListener('change', () => {
  const valor = lista.value;           // value del <option> elegido
  const indice = lista.selectedIndex;  // posición de la opción elegida
  const opcion = lista.options[indice];// el elemento <option> elegido
  const texto = opcion.text;           // el texto que ve el usuario
  mensaje.textContent = `${texto} (${valor})`;
});
```

***Listado 36*** *Acceso al valor seleccionado de una lista de selección*

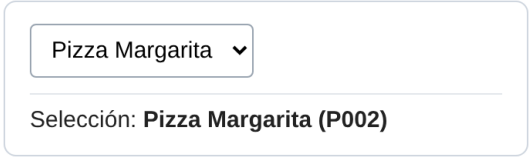



***Figura 8*** *El valor seleccionado se muestra bajo la lista mediante JavaScript*

**Valor y texto no son lo mismo**

`lista.value` contiene el **valor** de la opción (`P002`), no su texto (`Pizza Margarita`). El texto solo se obtiene a través del elemento `<option>`.

## 3.7 Lectura de los valores de listas de selección múltiples

Con el atributo `multiple` se definen listas en las que se pueden marcar varias opciones (Listado 37).

```
<select id="pizza" name="pizza" size="4" multiple>
  <option value="P001" selected>Pizza Salami</option>
  <option value="P002">Pizza Margarita</option>
  <option value="P003">Pizza Atún</option>
  <option value="P004">Pizza Mozzarella</option>
  <option value="P005">Pizza Hawaiana</option>
</select>
<div id="seleccion"></div>
```

***Listado 37*** *HTML de una lista de selección múltiple*

En este caso `selectedIndex` no sirve de mucho (solo devuelve la primera opción marcada). Se utiliza `selectedOptions`, una colección (`HTMLCollection`) con todas las opciones marcadas (Listado 38 y Figura 9).

```
const lista = document.getElementById('pizza');
const mensaje = document.getElementById('seleccion');

lista.addEventListener('change', () => {
  mensaje.replaceChildren(); // vacía el contenedor
  for (const opcion of lista.selectedOptions) { // solo las opciones marcadas
    const linea = document.createElement('div');
    linea.textContent = `${opcion.text} (${opcion.value})`;
    mensaje.append(linea);
  }
});
```

***Listado 38*** *Acceso a todos los valores seleccionados de una lista*

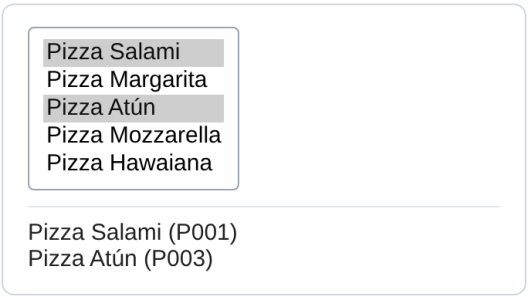



***Figura 9*** *Todos los valores seleccionados se muestran bajo la lista*

Si solo necesitas los valores, `Array.from()` convierte la colección en un array y permite transformarla a la vez (Listado 39).

```
const valores = Array.from(lista.selectedOptions, (opcion) => opcion.value);
console.log(valores); // p. ej., ['P001', 'P003']
```

***Listado 39*** *Obtener un array con los valores seleccionados*

### 3.8 Rellenar listas de selección con valores mediante JavaScript

A veces conviene generar las opciones de una lista dinámicamente. Se crean objetos `Option` y se añaden a la lista con `add()`. En el Listado 40 se recorre un array de objetos y se crea una opción por cada uno. El constructor admite hasta cuatro parámetros: texto visible, valor, si es la opción seleccionada por defecto (atributo `selected`) y si está seleccionada actualmente; los dos últimos son opcionales.

```
const lista = document.getElementById('pizza');
const pizzas = [
  { nombre: 'Pizza Salami', id: 'P001' },
  { nombre: 'Pizza Margarita', id: 'P002' },
  { nombre: 'Pizza Atún', id: 'P003' },
  { nombre: 'Pizza Mozzarella', id: 'P004' },
  { nombre: 'Pizza Hawaiana', id: 'P005' },
];

for (const pizza of pizzas) {
  // new Option(texto, valor, seleccionadaPorDefecto, seleccionada)
  lista.add(new Option(pizza.nombre, pizza.id));
}
```

***Listado 40*** *Añadir opciones a una lista de selección con JavaScript*

> **Selección de archivos**
>
> Si asignas el valor `file` al atributo `type` de un `<input>`, el formulario muestra un botón para seleccionar archivos (por ejemplo, para subir imágenes). La lectura de archivos con JavaScript se explica en un tema posterior, ya que utiliza una API de HTML5 que encaja mejor allí.

## 4 Envío y restablecimiento de formularios mediante programación

Un formulario con un botón `type="submit"` se envía solo, pero también se puede enviar y restablecer desde JavaScript. Para demostrarlo, en el formulario de acceso usamos dos botones `type="button"`, que por sí mismos no hacen nada (Listado 41 y Figura 10).

```
<form id="acceso" name="acceso" method="post" action="acceso.html">
  ...
  <!-- Dos botones que NO envían ni restablecen por sí solos -->
  <button type="button" id="entrar">Entrar</button>
  <button type="button" id="borrar">Borrar</button>
</form>
```

***Listado 41*** *Formulario de acceso con dos botones sin acción propia*

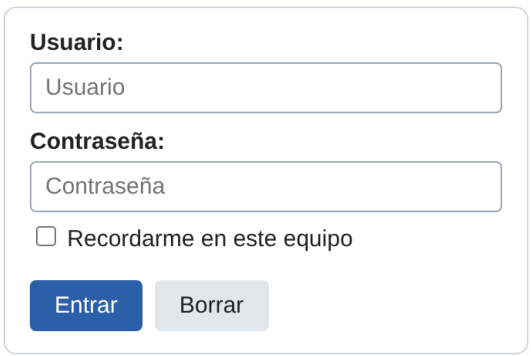



***Figura 10*** *El formulario de acceso ampliado*

El Listado 42 envía el formulario. Tienes dos métodos con una diferencia fundamental (Tabla 7): `requestSubmit()` actúa igual que pulsar un botón de envío (valida y lanza `submit`), mientras que `submit()` envía directamente.

| Forma de enviar | ¿Valida? | ¿Lanza el evento submit? |
|---|---|---|
| Botón `type="submit"` o tecla Intro | Sí | Sí |
| `formulario.requestSubmit()` | Sí | Sí |
| `formulario.submit()` | No | No |

***Tabla 7*** *Formas de enviar un formulario*

```
const formulario = document.getElementById('acceso');

document.getElementById('entrar').addEventListener('click', () => {
  formulario.requestSubmit(); // valida los campos y lanza el evento «submit»
  // formulario.submit();     // envía directamente, SIN validar y SIN evento «submit»
});
```

***Listado 42*** *Envío de un formulario mediante programación*

El Listado 43 restablece el formulario con `reset()`, que devuelve cada campo a su **valor inicial** (el que tiene en el HTML), no necesariamente a vacío.

```
document.getElementById('borrar').addEventListener('click', () => {
  formulario.reset(); // restablece los valores iniciales del HTML
});
```

***Listado 43*** *Restablecimiento de un formulario mediante programación*

> **El patrón moderno: submit + FormData**
>
> Lo habitual hoy es interceptar `submit`, cancelar el envío tradicional con `preventDefault()` y leer los datos con `FormData` para tratarlos con JavaScript (Listado 44). En temas posteriores los enviarás al servidor con `fetch()`.

```
formulario.addEventListener('submit', (evento) => {
  evento.preventDefault();                  // evitamos el envío tradicional
  const datos = new FormData(formulario);   // recoge todos los campos con atributo name
  console.log(datos.get('usuario'));        // un campo concreto
  console.log(Object.fromEntries(datos));   // todos, como objeto: { usuario: 'ana', ... }
});
```

***Listado 44*** *Leer los datos del formulario con FormData*

### FormData en detalle

`FormData` es un objeto que recoge los campos de un formulario como pares *nombre-valor*. Se crea con `new FormData(formulario)` y ofrece los métodos de la Tabla 8. Usaremos el formulario de pedido del Listado 45.

| Método | Qué hace |
|---|---|
| `get(nombre)` | Devuelve el valor del campo (el primero, si hay varios con el mismo nombre). |
| `getAll(nombre)` | Devuelve un array con **todos** los valores de ese nombre (por ejemplo, varias casillas). |
| `has(nombre)` | Indica si existe un campo con ese nombre. |
| `append(nombre, valor)` | Añade un dato, aunque ya exista otro con el mismo nombre. |
| `set(nombre, valor)` | Sustituye el valor del campo (o lo crea si no existe). |
| `delete(nombre)` | Elimina el campo. |

***Tabla 8*** *Métodos principales de FormData*

```
<form id="pedido">
  <input type="text" name="nombre" value="Ana">
  <select name="pizza">
    <option value="margarita">Margarita</option>
    <option value="cuatro-quesos">Cuatro quesos</option>
  </select>
  <label><input type="checkbox" name="extras" value="queso"> Queso</label>
  <label><input type="checkbox" name="extras" value="aceitunas"> Aceitunas</label>
  <button type="submit">Enviar</button>
</form>
```

***Listado 45*** *Formulario de pedido para practicar con FormData*

En el Listado 46 se leen los datos de distintas formas. `FormData` es **iterable**, así que se puede recorrer con `for...of` y desestructurar cada par.

```
const formulario = document.getElementById('pedido');

formulario.addEventListener('submit', (evento) => {
  evento.preventDefault();
  const datos = new FormData(formulario);

  console.log(datos.get('nombre'));      // 'Ana' (un campo)
  console.log(datos.getAll('extras'));   // ['queso', 'aceitunas'] (varios valores)
  console.log(datos.has('pizza'));       // true

  for (const [clave, valor] of datos) {  // recorrer todos los pares
    console.log(`${clave} = ${valor}`);
  }
});
```

***Listado 46*** *Leer los datos con get(), getAll() y for...of*

Los datos también se pueden modificar antes de usarlos (Listado 47). Observa que `Object.fromEntries()` solo conserva **el último** valor de los nombres repetidos: para las casillas `extras` queda `aceitunas`. Si necesitas todos, usa `getAll()`.

```
const datos = new FormData(formulario);

datos.append('origen', 'web');       // añade un dato (aunque el nombre ya exista)
datos.set('nombre', 'Luis');         // sustituye el valor del campo
datos.delete('pizza');               // elimina el campo

// Resultado: { nombre: 'Luis', extras: 'aceitunas', origen: 'web' }
console.log(Object.fromEntries(datos));
```

***Listado 47*** *Añadir, sustituir y eliminar datos*

**Qué datos recoge FormData**

`FormData` no recoge todos los controles: se **omiten** los campos sin atributo `name`, los deshabilitados (`disabled`), las casillas y los botones de opción **sin marcar** y los botones (salvo el que se indique, véase más abajo). Si un dato «no llega», revisa primero estas causas.

**FormData con el botón pulsado**

El constructor admite un segundo parámetro: el botón que provocó el envío. Así, si hay varios botones con `name` y `value` (por ejemplo, «Guardar» y «Enviar»), sabes cuál se pulsó (Listado 48). Se obtiene con `evento.submitter`.

```
formulario.addEventListener('submit', (evento) => {
  evento.preventDefault();
  // Con el segundo parámetro se incluye también el botón que provocó el envío
  const datos = new FormData(formulario, evento.submitter);
  console.log(datos.get('accion'));  // 'guardar' o 'enviar', según el botón pulsado
});
```

***Listado 48*** *Incluir el botón que provocó el envío*

> **Cuidado al elegir los nombres de los controles**
>
> Si un control del formulario se llama `submit` (`name="submit"`), tapa el método `submit()` del formulario y deja de funcionar (Listado 49 y Listado 50). Lo mismo ocurre con `reset` y con otros métodos del formulario. Usa `submit` o `reset` como nombre de un control solo si estás seguro de que no vas a usar esos métodos.

```
<button type="submit" name="submit">Enviar</button>
```

***Listado 49*** *Un botón llamado submit...*

```
formulario.submit(); // TypeError: formulario.submit is not a function
```

***Listado 50*** *...oculta el método submit() del formulario*

# 5 Validación de formularios

Validar significa comprobar que los datos introducidos son correctos antes de enviarlos. HTML5 incorpora opciones de validación nativas que, junto con JavaScript, permiten hacerlo con muy poco código.

> **Nativo**
>
> *Nativo*, en el contexto de HTML o de un lenguaje de programación, significa «integrado en el propio lenguaje». La validación nativa de HTML5 ya viene incluida y no hay que programarla con JavaScript.

## 5.1 Validación nativa de HTML5

El Listado 51 muestra un formulario de registro que activa la validación mediante atributos: el usuario y el correo son obligatorios (`required`); el usuario necesita al menos 3 caracteres (`minlength`); el correo solo admite direcciones de correo (`type="email"`); el sitio web, URL válidas (`type="url"`); y la edad, números entre 18 y 99 (`type="number"` con `min` y `max`). La Tabla 9 resume los atributos más habituales.

```
<form id="registro" name="registro" method="post" action="registro.html">
  <label for="usuario">Usuario:</label>
  <input type="text" id="usuario" name="usuario" required minlength="3">

  <label for="correo">Correo electrónico:</label>
  <input type="email" id="correo" name="correo" required>

  <label for="web">Sitio web:</label>
  <input type="url" id="web" name="web">

  <label for="edad">Edad:</label>
  <input type="number" id="edad" name="edad" min="18" max="99" value="18">

  <button type="submit">Registrarse</button>
</form>
```

***Listado 51*** *Ejemplo de validación nativa de HTML5*

| Atributo | Qué exige | Propiedad de validity que se activa |
|---|---|---|
| `required` | El campo no puede estar vacío. | `valueMissing` |
| `type="email" / "url"` | El valor debe tener formato de correo o URL. | `typeMismatch` |
| `min / max` | Valor numérico o fecha dentro de un rango. | `rangeUnderflow / rangeOverflow` |
| `minlength / maxlength` | Longitud mínima o máxima del texto. | `tooShort / tooLong` |
| `pattern="expresión"` | El valor debe cumplir una expresión regular (añade `title` para explicarla). | `patternMismatch` |
| `step` | El número debe ser múltiplo del paso indicado. | `stepMismatch` |

***Tabla 9*** *Atributos de validación nativa*

Sin una sola línea de JavaScript ya funciona la validación para estos casos. Si dejas vacío el correo e intentas enviar el formulario, el navegador impide el envío, muestra un mensaje de error (genérico) y aplica las reglas CSS de la pseudoclase `:invalid` (véase la Figura 11). El mensaje cambia según la causa del error.

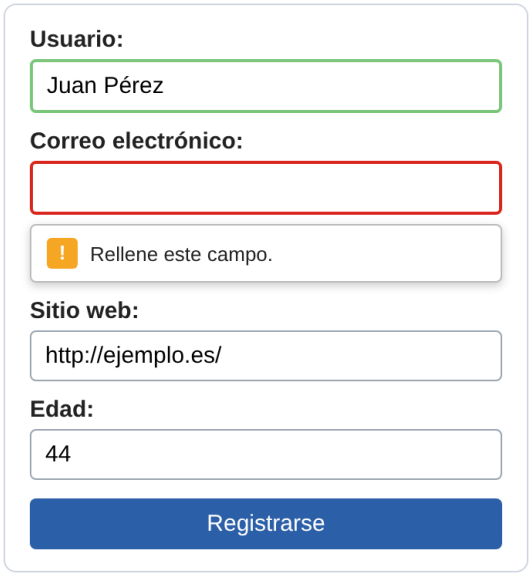



***Figura 11*** *Validación nativa de HTML5: campo vacío (representación del aviso del navegador)*

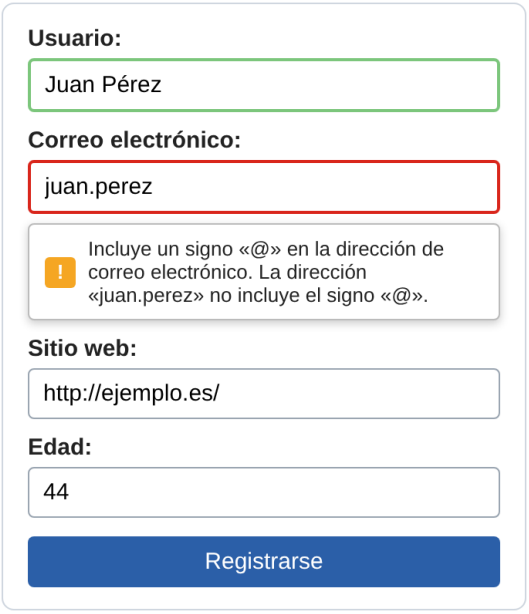



***Figura 12*** *El mensaje depende de la causa del error: aquí, un correo sin «@»*

Para los campos con valores válidos se aplican las reglas de `:valid`. La Figura 13 muestra el mismo formulario con datos correctos: ahora el envío funciona sin problemas.

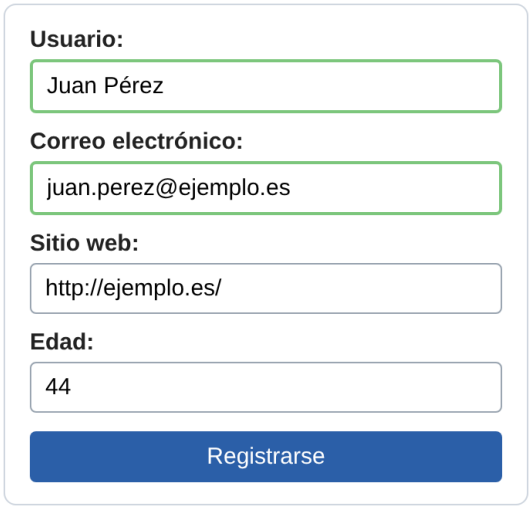



***Figura 13*** *El formulario con valores válidos*

### Pseudoclases CSS para formularios

Las pseudoclases `:required`, `:invalid` y `:valid` permiten dar estilo a los campos obligatorios, no válidos y válidos (Listado 52). Tienen un inconveniente: se aplican desde que se carga la página, por lo que el formulario aparece lleno de campos en rojo antes de que el usuario escriba nada.

> **Novedad CSS**
>
> Las pseudoclases `:user-invalid` y `:user-valid` (disponibles en los navegadores actuales) solo se aplican **después de que el usuario haya interactuado** con el campo. Son la opción recomendada para evitar errores prematuros.

```css
/* :valid / :invalid se aplican desde que se carga la página */
input:required:invalid { border: 2px solid red; }
input:required:valid   { border: 2px solid lightgreen; }

/* :user-invalid / :user-valid solo después de que el usuario haya interactuado */
input:user-invalid { border: 2px solid red; }
input:user-valid   { border: 2px solid lightgreen; }
```

***Listado 52*** *Pseudoclases CSS relacionadas con la validación nativa*

Pero, como ya habrás supuesto, la validación sin JavaScript es poco flexible. No puedes, por ejemplo, comprobar si un nombre de usuario ya está en uso (requiere una consulta a una base de datos), ni validar relaciones entre campos (que dos campos tengan el mismo valor), y el control sobre los mensajes de error y su aspecto es muy limitado.

## 5.2 La API de validación de restricciones

Gracias a la API de validación de restricciones (*Constraint Validation API*, definida en html.spec.whatwg.org), la validación nativa también se controla con JavaScript, lo que permite consultar el estado de validez y validar con mucha más flexibilidad. Define propiedades (Tabla 10) y métodos (Tabla 12) para formularios y controles.

| Propiedad | Descripción |
|---|---|
| `willValidate` | Indica si el campo se va a validar. Vale `true` por defecto y `false` si está deshabilitado (`disabled`) o es de un tipo que no se valida (por ejemplo, un botón `type="button"`). |
| `validity` | Objeto `ValidityState` con información detallada sobre la validez del valor (Tabla 11). |
| `validationMessage` | Mensaje de error que mostraría el navegador. Es una cadena vacía si el campo es válido. Varía según el navegador y el idioma. |

***Tabla 10*** *Propiedades relacionadas con la validación nativa*

La propiedad `validity` contiene varias propiedades booleanas que indican la causa del error: por ejemplo, `valueMissing` (falta un valor obligatorio), `tooShort` o `tooLong`, o `rangeUnderflow` y `rangeOverflow` (Tabla 11).

| Propiedad | Descripción |
|---|---|
| `valid` | El campo no tiene ningún error. |
| `valueMissing` | Es obligatorio y no tiene valor. |
| `typeMismatch` | El valor no cumple el tipo exigido (por ejemplo, un correo sin «@»). |
| `patternMismatch` | El valor no cumple el patrón (`pattern`). |
| `tooLong` | El valor es demasiado largo (`maxlength`). |
| `tooShort` | El valor es demasiado corto (`minlength`). |
| `rangeUnderflow` | El valor está por debajo del mínimo (`min`). |
| `rangeOverflow` | El valor está por encima del máximo (`max`). |
| `stepMismatch` | El valor no cumple el atributo `step`. |
| `badInput` | El navegador no puede interpretar lo escrito (por ejemplo, letras en un campo numérico). |
| `customError` | Se ha establecido un error propio con `setCustomValidity()`. |

***Tabla 11*** *Propiedades del objeto ValidityState*

El Listado 53 muestra algunas de estas propiedades al cambiar el campo de correo.

```
const campoCorreo = document.getElementById('correo');

campoCorreo.addEventListener('change', () => {
  console.log(campoCorreo.willValidate);          // true
  console.log(campoCorreo.validity);              // ValidityState { valueMissing: ..., ... }
  console.log(campoCorreo.validity.valueMissing); // ¿falta el valor?
  console.log(campoCorreo.validity.valid);        // ¿es válido?
  console.log(campoCorreo.validationMessage);     // mensaje del navegador ('' si es válido)
});
```

***Listado 53*** *Salida de los datos de validación*

Podemos hacerlo más práctico mostrando el mensaje de error **debajo del campo** (Figura 14). Para ello se añade un párrafo vacío tras el campo (Listado 54), se oculta cuando está vacío con la pseudoclase `:empty` (Listado 55) y se escribe en él `validationMessage` (Listado 56).

```
<label for="correo">Correo electrónico:</label>
<input type="email" id="correo" name="correo" required>
<p id="error-correo" class="error"></p>
```

***Listado 54*** *HTML con una zona para el mensaje de error*

```
.error { color: darkred; margin: 4px 0 12px; }
.error:empty { display: none; } /* sin mensaje, no ocupa espacio */
```

***Listado 55*** *CSS de la zona de error*

```
const campoCorreo = document.getElementById('correo');
const zonaError = document.getElementById('error-correo');

campoCorreo.addEventListener('change', () => {
  // validationMessage es '' cuando el campo es válido, así que no hace falta un if
  zonaError.textContent = campoCorreo.validationMessage;
});
```

***Listado 56*** *Mostrar el mensaje de validación debajo del campo*




***Figura 14*** *El mensaje se muestra bajo el campo en lugar del aviso predeterminado*

Además, la API ofrece tres métodos (Tabla 12): `checkValidity()` indica si un control (o todo el formulario) es válido; `reportValidity()` hace lo mismo y además muestra el mensaje del navegador; y `setCustomValidity()` permite definir un mensaje propio.

| Método | Descripción |
|---|---|
| `checkValidity()` | Se puede llamar en un control o en un formulario. Devuelve `true` si el control (o todos los controles del formulario) es válido. Si no lo es, se activa el evento `invalid`. |
| `reportValidity()` | Como `checkValidity()`, pero además muestra el mensaje de validación del navegador. |
| `setCustomValidity(mensaje)` | Establece un mensaje de error propio. Con una cadena vacía (`''`) el control vuelve a considerarse válido. |

***Tabla 12** Métodos relacionados con la validación nativa*

## 5.3 Mensajes de validación personalizados

`setCustomValidity()` permite validaciones que el HTML no puede expresar. Ampliamos el formulario con un segundo campo de correo (Listado 57) para evitar que el usuario escriba una dirección válida pero equivocada. La función `comprobarCorreos()` (Listado 58) se registra en ambos campos y comprueba si coinciden: si no, establece un mensaje de error; si coinciden, llama a `setCustomValidity('')` para quitarlo. Resultados en la Figura 15 y la Figura 16.

```
<label for="correo2">Repite el correo:</label>
<input type="email" id="correo2" name="correo2" required>
<p id="error-correo2" class="error"></p>
```

***Listado 57** Segundo campo de correo electrónico*

```
const correo = document.getElementById('correo');
const correo2 = document.getElementById('correo2');
const zonaError2 = document.getElementById('error-correo2');

function comprobarCorreos() {
  const mensaje = (correo.value === correo2.value) ? '' : 'Los correos deben coincidir.';
  correo2.setCustomValidity(mensaje); // '' significa «campo válido»
  zonaError2.textContent = correo2.validationMessage;
}

correo.addEventListener('input', comprobarCorreos);
correo2.addEventListener('input', comprobarCorreos);
```

***Listado 58** Comprobar si dos campos tienen el mismo valor*




***Figura 15*** *Validación imposible solo con HTML: los correos no coinciden*

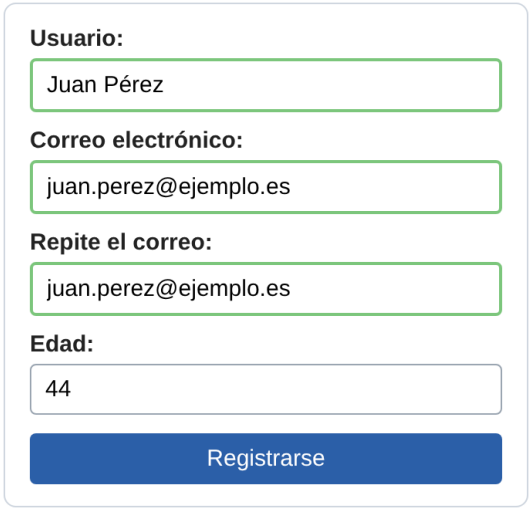



***Figura 16*** *La validación es correcta cuando ambos correos coinciden*

**No olvides limpiar el mensaje**

Si no vuelves a llamar a `setCustomValidity('')`, el campo seguirá considerándose no válido para siempre, aunque el usuario corrija el valor.

## 5.4 Decidir cuándo y cómo se muestran los errores

De forma predeterminada, la validación nativa se realiza *antes* de que se active el evento `submit`: el navegador muestra primero sus avisos y no te da opción de intervenir. Si prefieres controlar tú todo el proceso, establece la propiedad `noValidate` del formulario en `true`: así el navegador no valida ni muestra avisos, y tú decides qué hacer en el evento `submit`, con `checkValidity()` (Listado 59).

```
const formulario = document.getElementById('registro');
formulario.noValidate = true; // desactiva los avisos automáticos del navegador

formulario.addEventListener('submit', (evento) => {
  if (!formulario.checkValidity()) { // ¿son válidos TODOS los campos?
    evento.preventDefault();         // no se envía el formulario
    // aquí mostraríamos nuestros propios mensajes
  }
});
```

***Listado 59*** *Validar todos los campos al enviar el formulario*

Cuando un control no supera la validación se activa en él el evento `invalid`. Como **no burbujea**, para atender a todos los campos con un único escuchador se registra en la fase de captura (Listado 60). Con `preventDefault()` se evita el aviso del navegador y se muestra el mensaje propio.

```
// El evento invalid no burbujea: se escucha en la fase de captura (true)
formulario.addEventListener('invalid', (evento) => {
  evento.preventDefault(); // evita la burbuja de aviso del navegador
  const campo = evento.target;
  document.getElementById(`error-${campo.id}`).textContent = campo.validationMessage;
}, true);
```

***Listado 60*** *Mostrar un mensaje propio cuando un campo no es válido*

## 5.5 Ejemplo completo con módulos

Reunimos todo lo aprendido en un ejemplo con **módulos ES6**: `validaciones.js` exporta una función `mostrarError()` y `registro.js` la importa. Se valida cada campo al abandonarlo (delegando `focusout` en el formulario, sección 2) y todos los campos al enviar. Se asume que debajo de cada campo hay un `<p class="error" id="error-...">`, con el mismo `id` del campo precedido de `error-` (Listado 54).

```
// js/validaciones.js
export function mostrarError(campo) {
  const zonaError = document.getElementById(`error-${campo.id}`);
  if (zonaError) zonaError.textContent = campo.validationMessage;
}
```

***Listado 61*** *Módulo con la función de mostrar errores*

```
// js/registro.js
import { mostrarError } from './validaciones.js';

const formulario = document.getElementById('registro');
formulario.noValidate = true;

// Delegación: focusout burbujea, así que un solo escuchador sirve para todos los campos
formulario.addEventListener('focusout', (evento) => {
  if (evento.target.willValidate) mostrarError(evento.target);
});

formulario.addEventListener('submit', (evento) => {
  if (!formulario.checkValidity()) {
    evento.preventDefault();
    for (const campo of formulario.elements) {
      if (campo.willValidate) mostrarError(campo);
    }
  }
});
```

***Listado 62*** *Módulo principal: validación al salir de cada campo y al enviar*

```
<script type="module" src="js/registro.js"></script>
```

***Listado 63*** *Carga del módulo principal en el HTML*

# 6 Resumen

- **Eventos de formulario:** `input` (cada modificación), `change` (valor confirmado), `submit` (envío, se escucha en el `<form>`), `reset`, `invalid`; y `focus`/`blur` (no burbujean) frente a `focusin`/`focusout` (sí burbujean).
- **Eventos de interfaz:** `load`, `scroll` (mejor con `{ passive: true }`), `resize`, `beforeunload`, `visibilitychange`... Para saber si un elemento es visible, usa `IntersectionObserver`.
- **Delegación de eventos:** un único escuchador en un ancestro, `evento.target` + `closest()` para saber quién originó el evento. Menos código y funciona con elementos creados después.
- **Acceso a formularios y controles:** preferiblemente con `getElementById()` o `querySelector()`; también `formulario.elements.nombre`. `value` siempre es texto; `checked` en casillas; `selectedOptions` en listas múltiples.
- **Envío:** `requestSubmit()` valida y lanza `submit`; `submit()` no. Con `preventDefault()` y `FormData` se gestionan los datos con JavaScript. `reset()` restablece los valores iniciales.
- **Validación:** atributos HTML (`required`, `type`, `min`, `max`, `minlength`, `pattern`) + API de validación (`validity`, `validationMessage`, `checkValidity()`, `setCustomValidity()`) + pseudoclases CSS (`:user-invalid`).